

Title: A Unified Executors Proposal for C++

Authors: Jared Hoberock, jhoberock@nvidia.com
Michael Garland, mgarland@nvidia.com
Chris Kohlhoff, chris@kohlhoff.com
Chris Mysisen, mysen@google.com
Carter Edwards, hcedwar@sandia.gov
Gordon Brown, gordon@codeplay.com

Other Contributors: Hans Boehm, hboehm@google.com
Thomas Heller, thom.heller@gmail.com
Lee Howes, lwh@fb.com
Bryce Leibel, bryceleibel@gmail.com
Hartmut Kaiser, hartmut.kaiser@gmail.com
Bryce Leibel, bryceleibel@gmail.com
Gor Nishanov, gorn@microsoft.com
Thomas Rodgers, rodgert@twrogers.com
David Hollman, dshollm@sandia.gov
Michael Wong, michael@codeplay.com

Document Number: P0443R5

Date: 2018-02-12

Audience: SG1 - Concurrency and Parallelism, LEWG

Reply-to: sg1-exec@googlegroups.com

Abstract: This paper proposes a programming model for executors, which are modular components for creating execution. The design of this proposal is described in paper [P0761](#).

0.1 Changelog

0.1.1 Revision 5

Revision 5 of this proposal responds to feedback requested during the 2017 Albuquerque ISO C++ Standards Committee meeting and introduces changes which allow properties to better interoperate with polymorphic executor wrappers and also simplify `execution::require`'s behavior.

- Defined general property type requirements
- Elaborated specification of standard property types
- Simplified `execution::prefer`'s specification
- Enhanced polymorphic executor wrapper
 - Templated `execution::executor<SupportableProperties...>`
 - Introduced `prefer_only` property adaptor
- Responded to Albuquerque feedback
 - From SG1
 - Execution contexts are now optional properties of executors
 - Eliminated ill-specified caller-agent forward progress properties
 - Elaborated `Future`'s requirements to incorporate forward progress
 - Reworded operational semantics of execution functions to use similar language as the blocking properties
 - Elaborated `static_thread_pool`'s specification to guarantee that threads in the pool boost-block their work
 - Elaborated operational semantics of execution functions to note that forward progress guarantees are specific to the concrete executor type
 - From LEWG
 - Eliminated named `BaseExecutor` concept
 - Simplified general executor requirements
 - Enhanced the `OneWayExecutor` introductory paragraph
 - Eliminated `has_*_member` type traits
- Minor changes
 - Renamed TS namespace from `concurrency_v2` to `executors_v1`
 - Introduced `static_query_v` enabling static queries
 - Eliminated unused `property_value` trait

- Eliminated the names `allocator_wrapper_t` and `default_allocator`

0.1.2 Revision 4

- Specified the guarantees implied by `bulk_sequenced_execution`, `bulk_parallel_execution`, and `bulk_unsequenced_execution`

0.1.3 Revision 3

- Introduced `execution::query()` for executor property introspection
- Simplified the design of `execution::prefer()`
- `oneway`, `twoway`, `single`, and `bulk` are now `require()`-only properties
- Introduced properties allowing executors to opt into adaptations that add blocking semantics
- Introduced properties describing the forward progress relationship between caller and agents
- Various minor improvements to existing functionality based on prototyping

0.1.4 Revision 2

- Separated wording from explanatory prose, now contained in paper [P0761](#)
- Applied the simplification proposed by paper [P0688](#)

0.1.5 Revision 1

- Executor category simplification
- Specified executor customization points in detail
- Introduced new fine-grained executor type traits
 - Detectors for execution functions
 - Traits for introspecting cross-cutting concerns
 - Introspection of mapping of agents to threads
 - Introspection of execution function blocking behavior
- Allocator support for single agent execution functions
- Renamed `thread_pool` to `static_thread_pool`
- New introduction

0.1.6 Revision 0

- Initial design

1 Proposed Wording

1.0.1 Header `<execution>` synopsis

```
namespace std {
namespace experimental {
inline namespace executors_v1 {
namespace execution {

    // Customization points:

    namespace {
        constexpr unspecified require = unspecified;
        constexpr unspecified prefer = unspecified;
        constexpr unspecified query = unspecified;
    }

    // Customization point type traits:

    template<class Executor, class... Properties> struct can_require;
    template<class Executor, class... Properties> struct can_prefer;
    template<class Executor, class Property> struct can_query;

    template<class Executor, class... Properties>
        constexpr bool can_require_v = can_require<Executor, Properties...>::value;
    template<class Executor, class... Properties>
        constexpr bool can_prefer_v = can_prefer<Executor, Properties...>::value;
    template<class Executor, class Property>
        constexpr bool can_query_v = can_query<Executor, Property>::value;

    // Associated execution context property:
```

```

struct context_t;

constexpr context_t context;

// Directionality properties:

struct oneway_t;
struct twoway_t;
struct then_t;

constexpr oneway_t oneway;
constexpr twoway_t twoway;
constexpr then_t then;

// Cardinality properties:

struct single_t;
struct bulk_t;

constexpr single_t single;
constexpr bulk_t bulk;

// Blocking properties:

struct possibly_blocking_t;
struct always_blocking_t;
struct never_blocking_t;

constexpr possibly_blocking_t possibly_blocking;
constexpr always_blocking_t always_blocking;
constexpr never_blocking_t never_blocking;

// Properties to allow adaptation of blocking and directionality:

struct adaptable_blocking_t;
struct not_adaptable_blocking_t;

constexpr adaptable_blocking_t adaptable_blocking;
constexpr not_adaptable_blocking_t not_adaptable_blocking;

// Properties to indicate if submitted tasks represent continuations:

struct not_continuation_t;
struct continuation_t;

constexpr not_continuation_t not_continuation;
constexpr continuation_t continuation;

// Properties to indicate likely task submission in the future:

struct not_outstanding_work_t;
struct outstanding_work_t;

constexpr not_outstanding_work_t not_outstanding_work;
constexpr outstanding_work_t outstanding_work;

// Properties for bulk execution guarantees:

struct bulk_sequenced_execution_t;
struct bulk_parallel_execution_t;
struct bulk_unsequenced_execution_t;

constexpr bulk_sequenced_execution_t bulk_sequenced_execution;
constexpr bulk_parallel_execution_t bulk_parallel_execution;
constexpr bulk_unsequenced_execution_t bulk_unsequenced_execution;

// Properties for mapping of execution on to threads:

struct other_execution_mapping_t;
struct thread_execution_mapping_t;
struct new_thread_execution_mapping_t;

constexpr other_execution_mapping_t other_execution_mapping;
constexpr thread_execution_mapping_t thread_execution_mapping;
constexpr new_thread_execution_mapping_t new_thread_execution_mapping;

// Memory allocation properties:

```

```

template <typename ProtoAllocator>
struct allocator_t;

constexpr allocator_t allocator;

// Executor type traits:

template<class Executor> struct is_oneway_executor;
template<class Executor> struct is_twoway_executor;
template<class Executor> struct is_then_executor;
template<class Executor> struct is_bulk_oneway_executor;
template<class Executor> struct is_bulk_twoway_executor;
template<class Executor> struct is_bulk_then_executor;

template<class Executor> constexpr bool is_oneway_executor_v = is_oneway_executor<Executor>::value;
template<class Executor> constexpr bool is_twoway_executor_v = is_twoway_executor<Executor>::value;
template<class Executor> constexpr bool is_then_executor_v = is_then_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_oneway_executor_v = is_bulk_oneway_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_twoway_executor_v = is_bulk_twoway_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_then_executor_v = is_bulk_then_executor<Executor>::value;

template<class Executor, class T> struct executor_future;
template<class Executor> struct executor_shape;
template<class Executor> struct executor_index;

template<class Executor, class T> using executor_future_t = typename executor_future<Executor, T>::type;
template<class Executor> using executor_shape_t = typename executor_shape<Executor>::type;
template<class Executor> using executor_index_t = typename executor_index<Executor>::type;

// Polymorphic executor wrappers:

class bad_executor;

template <class... SupportableProperties>
class executor;
template<class Property> struct prefer_only;

} // namespace execution
} // inline namespace executors_v1
} // namespace experimental
} // namespace std

```

1.1 Requirements

1.1.1 Customization point objects

(The following text has been adapted from the draft Ranges Technical Specification.)

A *customization point object* is a function object (C++ Std, [function.objects]) with a literal class type that interacts with user-defined types while enforcing semantic requirements on that interaction.

The type of a customization point object shall satisfy the requirements of CopyConstructible (C++Std [copyconstructible]) and Destructible (C++Std [destructible]).

All instances of a specific customization point object type shall be equal.

Let t be a (possibly const) customization point object of type τ , and $\text{args} \dots$ be a parameter pack expansion of some parameter pack $\text{Args} \dots$. The customization point object t shall be callable as $t(\text{args} \dots)$ when the types of $\text{Args} \dots$ meet the requirements specified in that customization point object's definition. Otherwise, τ shall not have a function call operator that participates in overload resolution.

Each customization point object type constrains its return type to satisfy some particular type requirements.

The library defines several named customization point objects. In every translation unit where such a name is defined, it shall refer to the same instance of the customization point object.

[*Note:* Many of the customization points objects in the library evaluate function call expressions with an unqualified name which results in a call to a user-defined function found by argument dependent name lookup (C++Std [basic.lookup.argdep]). To preclude such an expression resulting in a call to unconstrained functions with the same name in namespace `std`, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace `std`. When the deleted overloads are viable, user-defined overloads must be more specialized (C++Std [temp.func.order]) to be used by a customization point object. --end note]

1.1.2 Future requirements

A type `F` meets the `Future` requirements for some value type `T` if `F` is `std::experimental::future<T>` (defined in the C++ Concurrency TS, ISO/IEC TS 19571:2016). [Note: This concept is included as a placeholder to be elaborated, with the expectation that the elaborated requirements for `Future` will expand the applicability of some executor customization points. --end note]

Forward progress guarantees are a property of the concrete `Future` type. [Note: `std::experimental::future<T>::wait()` blocks with forward progress guarantee delegation until the shared state is ready. --end note]

1.1.3 ProtoAllocator requirements

A type `A` meets the `ProtoAllocator` requirements if `A` is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where `U` is an object type. [Note: For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. --end note] No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

1.1.4 General requirements on executors

An executor type shall satisfy the requirements of `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `EqualityComparable` (C++Std [equalitycomparable]).

None of these concepts' operations, nor an executor type's swap operations, shall exit via an exception.

None of these concepts' operations, nor an executor type's associated execution functions, associated query functions, or other member functions defined in executor type requirements, shall introduce data races as a result of concurrent calls to those functions from different threads.

For any two (possibly const) values `x1` and `x2` of some executor type `X`, `x1 == x2` shall return `true` only if `x1.query(p) == x2.query(p)` for every property `p` where both `x1.query(p)` and `x2.query(p)` are well-formed and result in a non-void type that is `EqualityComparable` (C++Std [equalitycomparable]). [Note: The above requirements imply that `x1 == x2` returns `true` if `x1` and `x2` can be interchanged with identical effects. An executor may conceptually contain additional properties which are not exposed by a named property type that can be observed via `execution::query`; in this case, it is up to the concrete executor implementation to decide if these properties affect equality. Returning `false` does not necessarily imply that the effects are not identical. --end note]

An executor type's destructor shall not block pending completion of the submitted function objects. [Note: The ability to wait for completion of submitted function objects may be provided by the associated execution context. --end note]

1.1.5 OneWayExecutor requirements

The `OneWayExecutor` requirements specify requirements for executors which create execution agents. A type `x` satisfies the `OneWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

[Note: `OneWayExecutors` provides fire-and-forget semantics without a channel for awaiting the completion of a submitted function object and obtaining its result. --end note]

In the Table below, `x` denotes a (possibly const) executor object of type `X` and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.execute(f)</code>	<code>void</code>	Creates an execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>execute</code> .
		May block pending completion of <code>DECAY_COPY(std::forward<F>(f))()</code> .
		The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code> .

execute shall not propagate any exception thrown by `DECAY_COPY( std::forward<F>(f) )()` or any other function submitted to the executor. *[Note: The treatment of exceptions thrown by one-way submitted functions and the forward progress guarantee of execution agents created by one-way execution functions are specific to the concrete executor type. -- end note.]*

1.1.6 TwoWayExecutor requirements

The `TwoWayExecutor` requirements specify requirements for executors which create execution agents with a channel for awaiting the completion of a submitted function object and obtaining its result.

A type `X` satisfies the `TwoWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below, `x` denotes a (possibly const) executor object of type `X`, `f` denotes a function object of type `F&&` callable as `DECAY_COPY( std::forward<F>(f) )()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements, and `R` denotes the type of the expression `DECAY_COPY( std::forward<F>(f) )()`.

Expression	Return Type	Operational semantics
		Creates an execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>twoway_execute</code> .
		May block pending completion of <code>DECAY_COPY(std::forward<F>(f))()</code> .
<code>x.twoway_execute(f)</code>	A type that satisfies the Future requirements for the value type <code>R</code> .	The invocation of <code>twoway_execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code> .
		Stores the result of <code>DECAY_COPY(std::forward<F>(f))()</code> , or any exception thrown by <code>DECAY_COPY(std::forward<F>(f))()</code> , in the associated shared state of the resulting Future.

1.1.7 ThenExecutor requirements

The `ThenExecutor` requirements specify requirements for executors which create execution agents whose initiation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type `X` satisfies the `ThenExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below, `x` denotes a (possibly const) executor object of type `X`, `fut` denotes a future object satisfying the Future requirements, `f` denotes a function object of type `F&&` callable as `DECAY_COPY( std::forward<F>(f) )( fut )` and where `decay_t<F>` satisfies the `MoveConstructible` requirements, and `R` denotes the type of the expression `DECAY_COPY( std::forward<F>(f) )( fut )`.

Expression	Return Type	Operational semantics
		When <code>fut</code> is ready, creates an execution agent which invokes

		<code>DECAY_COPY(std::forward<F>(f))(fut)</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>then_execute</code> .
		May block pending completion of <code>DECAY_COPY(std::forward<F>(f))(fut)</code> .
<code>x.then_execute(f, fut)</code>	A type that satisfies the Future requirements for the value type <code>R</code> .	The invocation of <code>then_execute</code> synchronizes with (C++Std [<code>intro.multithread</code>]) the invocation of <code>f</code> .
		Stores the result of <code>DECAY_COPY(</code> <code>std::forward<F>(f))(fut)</code> , or any exception thrown by <code>DECAY_COPY(</code> <code>std::forward<F>(f))(fut)</code> , in the associated shared state of the resulting Future.

1.1.8 BulkOneWayExecutor requirements

The `BulkOneWayExecutor` requirements specify requirements for executors which create groups of execution agents in bulk from a single execution function, without a channel for awaiting the completion of the submitted function object invocations and obtaining their result. [*Note: That is, the executor provides fire-and-forget semantics. --end note*]

A type `X` satisfies the `BulkOneWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly `const`) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
		Invokes <code>sf()</code> on an unspecified execution agent to produce the value <code>s</code> . Creates a group of execution agents of shape <code>n</code> which invokes <code>DECAY_COPY(</code> <code>std::forward<F>(f))(i, s)</code> at most once for each value of <code>i</code> in the range <code>[0,n)</code> , with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>bulk_execute</code> .
		May block pending completion of one or more calls to <code>DECAY_COPY(</code> <code>std::forward<F>(f))(i, s)</code> .
<code>x.bulk_execute(f, n, sf)</code>	<code>void</code>	The invocation of <code>bulk_execute</code> synchronizes with (C++Std [<code>intro.multithread</code>]) the invocations of <code>f</code> .
		<code>bulk_execute</code> shall not propagate any exception thrown by <code>DECAY_COPY(</code> <code>std::forward<F>(f))(i, s)</code> or any other function submitted to the executor.

[*Note: The treatment of exceptions thrown by bulk one-way submitted functions and the forward progress guarantee of execution agents created by one-way execution functions is specific to the concrete executor type. -end note.*]

1.1.9 BulkTwoWayExecutor requirements

The `BulkTwoWayExecutor` requirements specify requirements for executors which create groups of execution agents in bulk from a single execution function with a channel for awaiting the completion of a submitted function object invoked by those execution agents and obtaining its result.

A type `X` satisfies the `BulkTwoWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly `const`) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- if `R` is non-void,
 - `r` denotes an object whose type is `R`,
 - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, r, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.
- if `R` is void,
 - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.bulk_twoway_execute(f, n, rf, sf)</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	If <code>R</code> is non-void, invokes <code>rf()</code> on an unspecified execution agent to produce the value <code>r</code>. Invokes <code>sf()</code> on an unspecified execution agent to produce the value <code>s</code>. Creates a group of execution agents of shape <code>n</code> which invokes <code>DECAY_COPY(std::forward<F>(f))(i, r, s)</code> if <code>R</code> is non-void, and otherwise invokes <code>DECAY_COPY(std::forward<F>(f))(i, s)</code>, at most once for each value of <code>i</code> in the range <code>[0, n)</code>, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>bulk_twoway_execute</code>. May block pending completion of one or more invocations of <code>f</code>. The invocation of <code>bulk_twoway_execute</code> synchronizes with (C++Std [intro.multithread]) the invocations of <code>f</code>. Once all invocations of <code>f</code> finish execution, stores <code>r</code>, or any exception thrown by an invocation of <code>f</code>, in the associated shared state of the resulting <code>Future</code>.

1.1.10 BulkThenExecutor requirements

The BulkThenExecutor requirements specify requirements for executors which create execution agents whose initiation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type `X` satisfies the BulkThenExecutor requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `fut` denotes a future object satisfying the Future requirements,
- `rf` denotes a CopyConstructible function object with zero arguments whose result type is `R`,
- `sf` denotes a CopyConstructible function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly const) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- if `R` is non-void,
 - `r` denotes an object whose type is `R`,
 - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, fut, r, s)` and where `decay_t<F>` satisfies the MoveConstructible requirements.
- if `R` is void,
 - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, fut, s)` and where `decay_t<F>` satisfies the MoveConstructible requirements.

Expression	Return Type	Operational semantics
<code>x.bulk_then_execute(f, n, fut, rf, sf)</code>	A type that satisfies the Future requirements for the value type <code>R</code> .	If <code>R</code> is non-void, invokes <code>rf()</code> on an unspecified execution agent to produce the value <code>r</code>. Invokes <code>sf()</code> on an unspecified execution agent to produce the value <code>s</code>. When <code>fut</code> is ready, creates a group of execution agents of shape <code>n</code> which invokes <code>DECAY_COPY(std::forward<F>(f))(i, fut, r, s)</code> if <code>R</code> is non-void, and otherwise invokes <code>DECAY_COPY(std::forward<F>(f))(i, fut, s)</code>, at most once for each value of <code>i</code> in the range <code>[0,n)</code>, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>bulk_then_execute</code>. May block pending completion of one or more invocations of <code>f</code>. The invocation of <code>bulk_then_execute</code> synchronizes with (C++Std [intro.multithread]) the invocations of <code>f</code>. Once all invocations of <code>f</code> finish execution, stores <code>r</code>, or any exception thrown by an invocation of <code>f</code>, in the associated shared state of the resulting Future.

1.2 Executor customization points

Executor customization points are functions which adapt an executor's properties. Executor customization points enable uniform use of executors in generic contexts.

When an executor customization point named *NAME* invokes a free execution function of the same name, overload resolution is performed in a context that includes the declaration `void NAME(auto&... args) = delete;`, where `sizeof...(args)` is the arity of the free execution function. This context also does not include a declaration of the executor customization point.

[*Note:* This provision allows executor customization points to call the executor's free, non-member execution function of the same name without recursion. *--end note*]

Whenever `std::experimental::executors_v1::execution::NAME(ARGS)` is a valid expression, that expression satisfies the syntactic requirements for the free execution function named *NAME* with arity `sizeof...(ARGS)` with that free execution function's semantics.

1.2.1 require

```
namespace {  
    constexpr unspecified require = unspecified;  
}
```

The name `require` denotes a customization point. The effect of the expression

`std::experimental::executors_v1::execution::require(E, P0, Pn...)` for some expressions *E* and *P0*, and where *Pn...* represents *N* expressions (where *N* is 0 or more), is equivalent to:

- If *N* == 0, *P0::is_requirable* is true, and the expression `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>> == decay_t<decltype(P0)>::value()` is a well-formed constant expression with value true, *E*.
- If *N* == 0, *P0::is_requirable* is true, and the expression `(E).require(P0)` is well-formed, `(E).require(P0)`.
- If *N* == 0, *P0::is_requirable* is true, and the expression `require(E, P0)` is well-formed, `require(E, P0)`.
- If *N* > 0 and the expression `std::experimental::executors_v1::execution::require(std::experimental::executors_v1::execution::require(E, P0), Pn...)` is well formed, `std::experimental::executors_v1::execution::require( std::experimental::executors_v1::execution::require(E, P0), Pn...)`.
- Otherwise, `std::experimental::executors_v1::execution::require(E, P0, Pn...)` is ill-formed.

1.2.2 prefer

```
namespace {  
    constexpr unspecified prefer = unspecified;  
}
```

The name `prefer` denotes a customization point. The effect of the expression

`std::experimental::executors_v1::execution::prefer(E, P0, Pn...)` for some expressions *E* and *P0*, and where *Pn...* represents *N* expressions (where *N* is 0 or more), is equivalent to:

- If *N* == 0, *P0::is_preferable* is true, and the expression `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>> == decay_t<decltype(P0)>::value()` is a well-formed constant expression with value true, *E*.
- If *N* == 0, *P0::is_preferable* is true, and the expression `(E).require(P0)` is well-formed, `(E).require(P0)`.
- If *N* == 0, *P0::is_preferable* is true, and the expression `prefer(E, P0)` is well-formed, `prefer(E, P0)`.
- If *N* == 0 and *P0::is_preferable* is true, *E*.
- If *N* > 0 and the expression `std::experimental::executors_v1::execution::prefer(std::experimental::executors_v1::execution::prefer(E, P0), Pn...)` is well formed, `std::experimental::executors_v1::execution::prefer( std::experimental::executors_v1::execution::prefer(E, P0), Pn...)`.
- Otherwise, `std::experimental::executors_v1::execution::require(E, P0, Pn...)` is ill-formed.

1.2.3 query

```
namespace {  
    constexpr unspecified query = unspecified;  
}
```

The name `query` denotes a customization point. The effect of the expression `std::experimental::executors_v1::execution::query(E, P)` for some expressions *E* and *P* is equivalent to:

- If the expression `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>>` is a well-formed constant expression, `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>>`.
- If the expression `(E).query(P0)` is well-formed, `(E).query(P0)`.
- If the expression `query(E, P0)` is well-formed, `query(E, P0)`.
- Otherwise, `std::experimental::executors_v1::execution::query(E, P)` is ill-formed.

1.2.4 Customization point type traits

```
template<class Executor, class... Properties> struct can_require;
template<class Executor, class... Properties> struct can_prefer;
template<class Executor, class Property> struct can_query;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
<pre>template<class T> struct can_require</pre>	The expression <code>std::experimental::executors_v1::execution::require(declval<const Executor>(), declval<Properties>()...)</code> is well formed.	$\mathbb{T}$ is a complete type.
<pre>template<class T> struct can_prefer</pre>	The expression <code>std::experimental::executors_v1::execution::prefer(declval<const Executor>(), declval<Properties>()...)</code> is well formed.	$\mathbb{T}$ is a complete type.
<pre>template<class T> struct can_query</pre>	The expression <code>std::experimental::executors_v1::execution::query(declval<const Executor>(), declval<Property>())</code> is well formed.	$\mathbb{T}$ is a complete type.

1.3 Executor properties

1.3.1 In general

An executor's behavior in generic contexts is determined by a set of executor properties, and each executor property imposes certain requirements on the executor.

Given an existing executor, a related executor with different properties may be created by calling the `require` member or non-member functions. These functions behave according the table below. In the table below, `x` denotes a (possibly `const`) executor object of type `x`, and `p` denotes a (possibly `const`) property object.

[*Note:* As a general design note properties which define a mutually exclusive pair, that describe an enabled or non-enabled behaviour follow the convention of having the same property name for both with the `not_` prefix to the property for the non-enabled behaviour. --end note]

Expression	Comments
<code>x.require(p)</code> <code>require(x,p)</code>	Returns an executor object with the requested property <code>p</code> added to the set. All other properties of the returned executor are identical to those of <code>x</code> , except where those properties are described below as being mutually exclusive to <code>p</code> . In this case, the mutually exclusive

properties are
implicitly removed
from the set
associated with the
returned executor.

The expression is
ill formed if an
executor is unable
to add the
requested
property.

The current value of an executor's properties can be queried by calling the `query` function. This function behaves according to the table below. In the table below, `x` denotes a (possibly `const`) executor object of type `x`, and `p` denotes a (possibly `const`) property object.

Expression	Comments
<code>x.query(p)</code>	Returns the current value of the requested property <code>p</code> . The expression is ill formed if an executor is unable to return the requested property.

1.3.2 Requirements on properties

A property type `P` shall provide:

- A nested constant expression named `is_requirable` of type `bool`, usable as `P::is_requirable`.
- A nested constant expression named `is_preferable` of type `bool`, usable as `P::is_preferable`.

[*Note*: These constants are used to determine whether the property can be used with the `require` and `prefer` customization points, respectively. *--end note*]

A property type `P` may provide a nested type `polymorphic_query_result_type`. [*Note*: When present, this type allows the property to be used with the polymorphic executor wrapper. *--end note*]

A property type `P` may provide:

- A nested variable template `static_query_v`, usable as `P::static_query_v<Executor>`. This may be conditionally present.
- A member function `value()`.

If both `static_query_v` and `value()` are present, they shall return the same type and this type shall satisfy the `EqualityComparable` requirements.

[*Note*: These are used to determine whether a `require` call would result in an identity transformation. *--end note*]

1.3.3 Query-only properties

1.3.3.1 Associated execution context property

```
struct context_t
{
    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type = any; // TODO: alternatively consider void*, or simply omitting the type.

    template<class Executor>
        static constexpr bool static_query_v
            = Executor::query(context_t());
};
```

```
};
```

The `context_t` property can be used only with `query`, which returns the **execution context** associated with the executor.

An execution context is a program object that represents a specific collection of execution resources and the **execution agents** that exist within those resources. Execution agents are units of execution, and a 1-to-1 mapping exists between an execution agent and an invocation of a callable function object submitted via the executor.

The value returned from `execution::query(e, context_t)`, where `e` is an executor, shall not change between calls.

1.3.4 Interface-changing properties

1.3.4.1 Directionality properties

```
struct oneway_t;
struct twoway_t;
struct then_t;
```

The directionality properties conform to the following specification:

```
struct S
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type = bool;

    template<class Executor>
        static constexpr bool static_query_v
            = see-below;

    static constexpr bool value() const { return true; }
};
```

Property	Requirements
oneway_t	The executor type satisfies the <code>OneWayExecutor</code> OR <code>BulkOneWayExecutor</code> requirements.
twoway_t	The executor type satisfies the <code>TwoWayExecutor</code> OR <code>BulkTwoWayExecutor</code> requirements.
then_t	The executor type satisfies the <code>ThenExecutor</code> OR <code>BulkThenExecutor</code> requirements.

`S::static_query_v<Executor>` is true if and only if `Executor` fulfills `S`'s requirements.

The `oneway_t`, `twoway_t` and `then_t` properties are not mutually exclusive.

1.3.4.1.1 twoway_t customization points

In addition to conforming to the above specification, the `twoway_t` property provides the following customization:

```
struct twoway_t
{
    template<class Executor>
        friend see-below require(Executor ex, twoway_t);
};
```

If the executor has the `oneway_t` and `adaptable_blocking_t` properties, this customization returns an adapter that implements the `twoway` property and its requirements.

Returns: A value `e1` of type `E1` that holds a copy of `ex`. If `Executor` satisfies the `OneWayExecutor` requirements, `E1` shall satisfy the `OneWayExecutor` requirements by providing member functions `require`, `query`, and `execute` that forward to the corresponding member functions of the copy of `ex`, if present, and `E1` shall satisfy the `TwoWayExecutor` requirements by implementing `twoway_execute` in terms of `execute`. Similarly, if `Executor` satisfies the `BulkOneWayExecutor` requirements, `E1` shall satisfy the `BulkOneWayExecutor` requirements by providing member functions `require`, `query`, and `bulk_execute` that forward to the

corresponding member functions of the copy of `ex`, if present, and `E1` shall satisfy the `BulkTwoWayExecutor` requirements by implementing `bulk_twoway_execute` in terms of `bulk_execute`. For some type `T`, the type yielded by `executor_future_t<E1, T>` is `std::experimental::future<T>`. `e1` has the same executor properties as `ex`, except for the addition of the `twoway_t` property.

Remarks: This function shall not participate in overload resolution unless `is_twoway_executor_v<Executor> || is_bulk_twoway_executor_v<Executor>` is false, `is_oneway_executor_v<Executor> || is_bulk_oneway_executor_v<Executor>` is true, and `adaptable_blocking_t::static_query_v<Executor>` is a constant expression with value true.

1.3.4.2 Cardinality properties

```
struct single_t;
struct bulk_t;
```

The cardinality properties conform to the following specification:

```
struct S
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type = bool;

    template<class Executor>
        static constexpr bool static_query_v
            = see-below;

    static constexpr bool value() const { return true; }
};
```

Property	Requirements
single_t	The executor type satisfies the <code>OneWayExecutor</code> , <code>TwoWayExecutor</code> , OR <code>ThenExecutor</code> requirements.
bulk_t	The executor type satisfies the <code>BulkOneWayExecutor</code> , <code>BulkTwoWayExecutor</code> , OR <code>BulkThenExecutor</code> requirements.

`S::static_query_v<Executor>` is true if and only if `Executor` fulfills `S`'s requirements.

The `single_t` and `bulk_t` properties are not mutually exclusive.

1.3.4.2.1 bulk_t customization points

In addition to conforming to the above specification, the `bulk_t` property provides the following customization:

```
struct bulk_t
{
    template<class Executor>
        friend see-below require(Executor ex, bulk_t);
};
```

If the executor has the `oneway_t` property, this customization returns an adapter that implements the `bulk` property and its requirements.

```
template<class Executor>
    friend see-below require(Executor ex, bulk_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. If `Executor` satisfies the `OneWayExecutor` requirements, `E1` shall satisfy the `OneWayExecutor` requirements by providing member functions `require`, `query`, and `execute` that forward to the corresponding member functions of the copy of `ex`, if present, and `E1` shall satisfy the `BulkExecutor` requirements by implementing `bulk_execute` in terms of `execute`. If `Executor` also satisfies the `TwoWayExecutor` requirements, `E1` shall satisfy the `TwoWayExecutor` requirements by providing the member function `twoway_execute` that forwards to the corresponding member function of the copy of `ex`, if present, and `E1` shall satisfy the `BulkTwoWayExecutor` requirements by implementing `bulk_twoway_execute` in terms of `bulk_execute`. `e1` has the same executor properties as `ex`, except for the addition of the `bulk_t` property.

Remarks: This function shall not participate in overload resolution unless `is_bulk_oneway_executor_v<Executor> || is_bulk_twoway_executor_v<Executor>` is false, and `is_oneway_executor_v<Executor>` is true.

1.3.5 Behavioral properties

Behavioral properties conform to the following specification:

```
struct S
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;

    using polymorphic_query_result_type = bool;

    template<class Executor>
        static constexpr bool static_query_v
            = Executor::query(S());

    static constexpr bool value() const { return true; }
};
```

The result of the query customization point for a behavioral property is `true` if the property is present in the executor, and `false` if it is not.

1.3.5.1 Blocking properties

```
struct possibly_blocking_t;
struct always_blocking_t;
struct never_blocking_t;
```

Property	Requirements
possibly_blocking_t	A call to an executor's execution function may block pending completion of one or more of the execution agents created by that execution function.
always_blocking_t	A call to an executor's execution function shall block until completion of all execution agents created by that execution function.
never_blocking_t	A call to an executor's execution function shall not block pending completion of the execution agents created by that execution function.

The `possibly_blocking_t`, `always_blocking_t` and `never_blocking_t` properties are mutually exclusive.

[*Note:* The guarantees of `possibly_blocking_t`, `always_blocking_t` and `never_blocking_t` implies the relationships: `possibly_blocking < always_blocking` and `possibly_blocking < never_blocking` *--end note*]

The value returned from `execution::query(e, p)` shall not change between calls unless `e` is assigned another executor which has a different value for `p`. The value returned from `execution::query(e, p1)` and a subsequent call `execution::query(e1, p1)` where: `* p1` is `execution::possibly_blocking`, `execution::always_blocking` OR `execution::never_blocking`, and `* e1` is the result of `execution::require(e, p2)` OR `execution::prefer(e, p2)`, shall compare equal unless: `* p2` is `execution::possibly_blocking`, `execution::always_blocking` OR `execution::never_blocking`, and `* p1` and `p2` are different types.

1.3.5.1.1 possibly_blocking_t customization points

In addition to conforming to the above specification, the `possibly_blocking_t` property provides the following customization:

```
struct possibly_blocking_t
{
    template<class Executor>
        friend constexpr bool query(const Executor& ex, possibly_blocking_t);
};
```

This customization automatically enables the `possibly_blocking_t` property for all executors that do not otherwise support the `always_blocking_t` or `never_blocking_t` properties. [Note: That is, all executors are treated as possibly blocking unless otherwise specified. --end note]

```
template<class Executor>
    friend constexpr bool query(const Executor& ex, possibly_blocking_t);
```

Returns: true.

Remarks: This function shall not participate in overload resolution unless `can_query_v<Executor, always_blocking_t> || can_query_v<Executor, never_blocking_t>` is false.

1.3.5.1.2 always_blocking_t customization points

In addition to conforming to the above specification, the `always_blocking_t` property provides the following customization:

```
struct always_blocking_t
{
    template<class Executor>
        friend see-below require(Executor ex, always_blocking_t);
};
```

If the executor has the `adaptable_blocking_t` property, this customization uses an adapter to implement the `always_blocking_t` property.

```
template<class Executor>
    friend see-below require(Executor ex, always_blocking_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. If `Executor` satisfies the `OneWayExecutor` requirements, `E1` shall satisfy the `OneWayExecutor` requirements by providing member functions `require`, `query`, and `execute` that forward to the corresponding member functions of the copy of `ex`. If `Executor` satisfies the `TwoWayExecutor` requirements, `E1` shall satisfy the `TwoWayExecutor` requirements by providing member functions `require`, `query`, and `twoway_execute` that forward to the corresponding member functions of the copy of `ex`. If `Executor` satisfies the `BulkOneWayExecutor` requirements, `E1` shall satisfy the `BulkOneWayExecutor` requirements by providing member functions `require`, `query`, and `bulk_execute` that forward to the corresponding member functions of the copy of `ex`. If `Executor` satisfies the `BulkTwoWayExecutor` requirements, `E1` shall satisfy the `BulkTwoWayExecutor` requirements by providing member functions `require`, `query`, and `bulk_twoway_execute` that forward to the corresponding member functions of the copy of `ex`. In addition, `E1` provides an overload of `require` such that `e1.require(always_blocking)` returns a copy of `e1`, an overload of `query` such that `e1.query(always_blocking)` returns `true`, and all functions `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute` shall block the calling thread until the submitted functions have finished execution. `e1` has the same executor properties as `ex`, except for the addition of the `always_blocking_t` property, and removal of `never_blocking_t` and `possibly_blocking_t` properties if present.

Remarks: This function shall not participate in overload resolution unless `adaptable_blocking_t::static_query_v<Executor>` is true.

1.3.5.2 Properties to indicate if blocking and directionality may be adapted

```
struct adaptable_blocking_t;
struct not_adaptable_blocking_t;
```

Property	Requirements
adaptable_blocking_t	The require customization point may adapt the executor to add the <code>two_way_t</code> or <code>always_blocking_t</code> properties.
	The require customization point may not adapt the executor
not_adaptable_blocking_t	

to add the
two_way_t or
always_blocking_t
properties.

The `not_adaptable_blocking_t` and `adaptable_blocking_t` properties are mutually exclusive.

[*Note:* The `two_way_t` property is included here as the `require customization` point's `two_way_t` adaptation is specified in terms of `std::experimental::future`, and that template supports blocking wait operations. *--end note*]

The value returned from `execution::query(e, p)` shall not change between calls unless `e` is assigned another executor which has a different value for `p`. The value returned from `execution::query(e, p1)` and a subsequent call `execution::query(e1, p1)` where: * `p1` is `execution::adaptable_blocking` or `execution::not_adaptable_blocking`, and * `e1` is the result of `execution::require(e, p2)` or `execution::prefer(e, p2)`, shall compare equal unless: * `p2` is `execution::adaptable_blocking` or `execution::not_adaptable_blocking`, and * `p1` and `p2` are different types.

1.3.5.2.1 adaptable_blocking_t customization points

In addition to conforming to the above specification, the `adaptable_blocking_t` property provides the following customization:

```
struct adaptable_blocking_t
{
    template<class Executor>
        friend see-below require(Executor ex, adaptable_blocking_t);
};
```

This customization uses an adapter to implement the `adaptable_blocking_t` property.

```
template<class Executor>
    friend see-below require(Executor ex, adaptable_blocking_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. If `Executor` satisfies the `OneWayExecutor` requirements, `E1` shall satisfy the `OneWayExecutor` requirements by providing member functions `require`, `query`, and `execute` that forward to the corresponding member functions of the copy of `ex`. If `Executor` satisfies the `TwoWayExecutor` requirements, `E1` shall satisfy the `TwoWayExecutor` requirements by providing member functions `require`, `query`, and `twoway_execute` that forward to the corresponding member functions of the copy of `ex`. If `Executor` satisfies the `BulkOneWayExecutor` requirements, `E1` shall satisfy the `BulkOneWayExecutor` requirements by providing member functions `require`, `query`, and `bulk_execute` that forward to the corresponding member functions of the copy of `ex`. If `Executor` satisfies the `BulkTwoWayExecutor` requirements, `E1` shall satisfy the `BulkTwoWayExecutor` requirements by providing member functions `require`, `query`, and `bulk_twoway_execute` that forward to the corresponding member functions of the copy of `ex`. In addition, `adaptable_blocking_t::static_query_v<E1>` is true, and `e1.require(not_adaptable_blocking)` yields a copy of `ex`. `e1` has the same executor properties as `ex`, except for the addition of the `adaptable_blocking_t` property.

1.3.5.3 Properties to indicate if submitted tasks represent continuations

```
struct continuation_t;
struct not_continuation_t;
```

Property	Requirements
<code>continuation_t</code>	Function objects submitted through the executor represent continuations of the caller. If the caller is a lightweight execution agent managed by the executor or its associated execution context, the execution of the submitted function object may be deferred until the caller completes. Function objects

submitted through
not_continuation_t the executor do not
represent
continuations of the
caller.

The not_continuation_t and continuation_t properties are mutually exclusive.

The value returned from execution::query(e, p) shall not change between calls unless e is assigned another executor which has a different value for p. The value returned from execution::query(e, p1) and a subsequent call execution::query(e1, p1) where: * p1 is execution::continuation OR execution::not_continuation, and * e1 is the result of execution::require(e, p2) OR execution::prefer(e, p2), shall compare equal unless: * p2 is execution::continuation OR execution::not_continuation, and * p1 and p2 are different types.

1.3.5.4 Properties to indicate likely task submission in the future

```
struct not_outstanding_work_t;  
struct outstanding_work_t;
```

Property	Requirements
outstanding_work_t	The existence of the executor object represents an indication of likely future submission of a function object. The executor or its associated execution context may choose to maintain execution resources in anticipation of this submission.
not_outstanding_work_t	The existence of the executor object does not indicate any likely future submission of a function object.

The not_outstanding_work_t and outstanding_work_t properties are mutually exclusive.

[*Note:* The outstanding_work_t and not_outstanding_work_t properties are use to communicate to the associated execution context intended future work submission on the executor. The intended effect of the properties is the behavior of execution context's facilities for awaiting outstanding work; specifically whether it considers the existance of the executor object with the outstanding_work_t property enabled outstanding work when deciding what to wait on. However this will be largely defined by the execution context implementation. It is intended that the execution context will define its wait facilities and on-destruction behaviour and provide an interface for querying this. An initial work towards this is included in P0737r0. *--end note*]

The value returned from execution::query(e, p) shall not change between calls unless e is assigned another executor which has a different value for p. The value returned from execution::query(e, p1) and a subsequent call execution::query(e1, p1) where: * p1 is execution::outstanding_work OR execution::not_outstanding_work, and * e1 is the result of execution::require(e, p2) OR execution::prefer(e, p2), shall compare equal unless: * p2 is execution::outstanding_work OR execution::not_outstanding_work, and * p1 and p2 are different types.

1.3.5.5 Properties for bulk execution guarantees

These properties communicate the forward progress and ordering guarantees of execution agents with respect to other agents within the same bulk submission.

```
struct bulk_sequenced_execution_t;
```

```
struct bulk_parallel_execution_t;
struct bulk_unsequenced_execution_t;
```

Property	Requirements
bulk_sequenced_execution_t	Execution agents within the same bulk execution may not be parallelized.
bulk_parallel_execution_t	Execution agents within the same bulk execution may be parallelized.
bulk_unsequenced_execution_t	Execution agents within the same bulk execution may be parallelized and vectorized.

Execution agents created by executors with the `bulk_sequenced_execution_t` property execute in sequence in lexicographic order of their indices.

Execution agents created by executors with the `bulk_parallel_execution_t` property execute with a parallel forward progress guarantee. Any such agents executing in the same thread of execution are indeterminately sequenced with respect to each other. [*Note: It is the caller's responsibility to ensure that the invocation does not introduce data races or deadlocks. --end note*]

Execution agents created by executors with the `bulk_unsequenced_execution_t` property may execute in an unordered fashion. Any such agents executing in the same thread of execution are unsequenced with respect to each other. [*Note: This means that multiple execution agents may be interleaved on a single thread of execution, which overrides the usual guarantee from [intro.execution] that function executions do not interleave with one another. --end note*]

[*Editorial note:* The descriptions of these properties were ported from [algorithms.parallel.user]. The intention is that a future standard will specify execution policy behavior in terms of the fundamental properties of their associated executors. We did not include the accompanying code examples from [algorithms.parallel.user] because the examples seem easier to understand when illustrated by `std::for_each`. --end editorial note]

[*Note:* The guarantees provided by these properties implies the relationship: `bulk_unsequenced_execution < bulk_parallel_execution < bulk_sequenced_execution`. --end note]

[*Editorial note:* The note above is intended to describe when one bulk executor can be substituted for another and provide the required semantics. For example, if a user needs sequenced execution, then only an executor with the `bulk_sequenced_execution_t` property will do. On the other hand, if a user only needs `bulk_unsequenced_execution_t`, then an executor with any of the above properties will suffice. --end editorial note]

The `bulk_unsequenced_execution_t`, `bulk_parallel_execution_t`, and `bulk_sequenced_execution_t` properties are mutually exclusive.

The value returned from `execution::query(e, p)` shall not change between calls unless `e` is assigned another executor which has a different value for `p`. The value returned from `execution::query(e, p1)` and a subsequent call `execution::query(e1, p1)` where: `* p1` is `execution::bulk_unsequenced_execution`, `execution::bulk_parallel_execution` OR `execution::bulk_sequenced_execution`, and `* e1` is the result of `execution::require(e, p2)` OR `execution::prefer(e, p2)`, shall compare equal unless: `* p2` is `execution::bulk_unsequenced_execution`, `execution::bulk_parallel_execution` OR `execution::bulk_sequenced_execution`, and `* p1` and `p2` are different types.

1.3.5.6 Properties for mapping of execution on to threads

```
struct other_execution_mapping_t;
struct thread_execution_mapping_t;
struct new_thread_execution_mapping_t;
```

Property	Requirements
	Mapping of each execution

other_execution_mapping_t	agent created by the executor is implementation defined.
thread_execution_mapping_t	Execution agents created by the executor are mapped onto threads of execution.
new_thread_execution_mapping_t	Each execution agent created by the executor is mapped onto a new thread of execution.

The other_execution_mapping_t, thread_execution_mapping_t and new_thread_execution_mapping_t properties are mutually exclusive.

[*Note:* The guarantees of other_execution_mapping_t, thread_execution_mapping_t and new_thread_execution_mapping_t implies the relationship: other_execution_mapping_t < thread_execution_mapping_t < new_thread_execution_mapping_t *--end note*]

[*Note:* A mapping of an execution agent onto a thread of execution implies the agent executes as-if on a std::thread. Therefore, the facilities provided by std::thread, such as thread-local storage, are available. new_thread_execution_mapping_t provides stronger guarantees, in particular that thread-local storage will not be shared between execution agents. *--end note*]

The value returned from execution::query(e, p) shall not change between calls unless e is assigned another executor which has a different value for p. The value returned from execution::query(e, p1) and a subsequent call execution::query(e1, p1) where: * p1 is execution::other_execution_mapping, execution::thread_execution_mapping OR execution::new_thread_execution_mapping, and * e1 is the result of execution::require(e, p2) OR execution::prefer(e, p2), shall compare equal unless: * p2 is execution::other_execution_mapping, execution::thread_execution_mapping OR execution::new_thread_execution_mapping, and * p1 and p2 are different types.

1.3.6 Properties for customizing memory allocation

```
template <typename ProtoAllocator>
struct allocator_t;
```

The allocator_t property conforms to the following specification:

```
template <typename ProtoAllocator>
struct allocator_t
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;

    template<class Executor>
    static constexpr bool static_query_v
        = Executor::query(allocator_t);

    template <typename OtherProtoAllocator>
    allocator_t<OtherProtoAllocator> operator()(const OtherProtoAllocator &a) const {
        return allocator_t<OtherProtoAllocator>{a};
    }

    static constexpr bool value() const { return a; }
};
```

Property	Requirements
allocator_t<ProtoAllocator>	Result of allocator_t<void>::operator(OtherProtoAllocator). The executor type satisfies the OneWayExecutor, TwoWayExecutor, Or ThenExecutor requirements. The executor implementation shall use the encapsulated allocator to allocate any memory required to store the submitted function object.

allocator_t<void>

Specialisation of allocator_t<ProtoAllocator>. The executor type satisfies the OneWayExecutor, TwoWayExecutor, Or ThenExecutor requirements. The executor implementation shall use an implementation defined default allocator to allocate any memory required to store the submitted function object.

Remarks: operator(OtherProtoAllocator) and value() shall not participate in overload resolution unless ProtoAllocator is void.

Postconditions: alloc.value() returns a, where alloc is the result of allocator(a).

[*Note:* Where the allocator_t is queryable, it must be accepted as both allocator_t<ProtoAllocator> and allocator_t<void>. *--end note*]

[*Note:* As the allocator_t<ProtoAllocator> property enapsulates a value which can be set and queried, it is required to be implemented such that it is callable with the OtherProtoAllocator parameter where the customization points accepts the result of allocator_t<void>::operator(OtherProtoAllocator); allocator_t<OtherProtoAllocator> and is passable as an instance where the customization points accept an instance of allocator_t<void>. *--end note*]

[*Note:* It is permitted for an allocator provided via allocator_t<void>::operator(OtherProtoAllocator) property to be the same type as the default allocator provided by the implementation. *--end note*]

The value returned from execution::query(e, p) shall not change between calls unless e is assigned another executor which has a different value for p. The value returned from execution::query(e, p1) and a subsequent call execution::query(e1, p1) where: * p1 is execution::allocator OR execution::allocator(ProtoAllocator), and e1 is the result of execution::require(e, p2) OR execution::prefer(e, p2), shall compare equal unless: * p2 is execution::allocator OR execution::allocator(ProtoAllocator).

1.4 Executor type traits

1.4.1 Determining that a type satisfies executor type requirements

```
template<class T> struct is_oneway_executor;
template<class T> struct is_twoway_executor;
template<class T> struct is_then_executor;
template<class T> struct is_bulk_oneway_executor;
template<class T> struct is_bulk_twoway_executor;
template<class T> struct is_bulk_then_executor;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a UnaryTypeTrait (C++Std [meta.rqmts]) with a BaseCharacteristic of true_type if the corresponding condition is true, otherwise false_type.

Template	Condition	Preconditions
template<class T> struct is_oneway_executor	τ meets the syntactic requirements for OneWayExecutor.	τ is a complete type.
template<class T> struct is_twoway_executor	τ meets the syntactic requirements for TwoWayExecutor.	τ is a complete type.
template<class T> struct is_then_executor	τ meets the syntactic requirements for ThenExecutor.	τ is a complete type.
template<class T> struct is_bulk_oneway_executor	τ meets the syntactic requirements for BulkOneWayExecutor.	τ is a complete type.
template<class T> struct is_bulk_twoway_executor	τ meets the syntactic requirements for BulkTwoWayExecutor.	τ is a complete type.
template<class T> struct is_bulk_then_executor	τ meets the syntactic requirements for BulkThenExecutor.	τ is a complete type.

1.4.2 Associated future type

```
template<class Executor, class T>
struct executor_future
{
    using type = decltype(execution::require(declval<const Executor&>(), execution::twoway).twoway_execute(declval<T(*)>()));
};
```

1.4.3 Associated shape type

```
template<class Executor>
struct executor_shape
{
private:
    // exposition only
    template<class T>
    using helper = typename T::shape_type;

public:
    using type = std::experimental::detected_or_t<
        size_t, helper, decltype(execution::require(declval<const Executor>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "shape type must be an integral type");
};
```

1.4.4 Associated index type

```
template<class Executor>
struct executor_index
{
private:
    // exposition only
    template<class T>
    using helper = typename T::index_type;

public:
    using type = std::experimental::detected_or_t<
        executor_shape_t<Executor>, helper, decltype(execution::require(declval<const Executor>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "index type must be an integral type");
};
```

1.5 Polymorphic executor wrappers

[Commentary: The polymorphic executor wrapper class has been written such that it could be separated from the rest of the proposal if necessary.]

In several places in this section the operation `CONTAINS_PROPERTY(p, pn)` is used. All such uses mean `std::disjunction_v<std::is_same<p, pn>...>`.

1.5.1 Class `bad_executor`

An exception of type `bad_executor` is thrown by executor member functions `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute` when the executor object has no target.

```
class bad_executor : public exception
{
public:
    // constructor:
    bad_executor() noexcept;
};
```

1.5.1.1 `bad_executor` constructors

```
bad_executor() noexcept;
```

Effects: Constructs a `bad_executor` object.

Postconditions: `what()` returns an implementation-defined NTBS.

1.5.2 Class template `executor`

The executor class template provides a polymorphic wrapper for executor types.

```
template <class... SupportableProperties>
class executor
{

```

```

public:
    // construct / copy / destroy:

    executor() noexcept;
    executor(nullptr_t) noexcept;
    executor(const executor& e) noexcept;
    executor(executor&& e) noexcept;
    template<class Executor> executor(Executor e);
    template<class... OtherSupportableProperties> executor(executor<OtherSupportableProperties...> e);

    executor& operator=(const executor& e) noexcept;
    executor& operator=(executor&& e) noexcept;
    executor& operator=(nullptr_t) noexcept;
    template<class Executor> executor& operator=(Executor e);

    ~executor();

    // executor modifiers:

    void swap(executor& other) noexcept;

    // executor operations:

    template <class Property>
    executor require(Property) const;

    template <class Property>
    executor query(Property) const;

    template<class Function>
    void execute(Function&& f) const;

    template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const

    template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

    template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;

    // executor capacity:

    explicit operator bool() const noexcept;

    // executor target access:

    const type_info& target_type() const noexcept;
    template<class Executor> Executor* target() noexcept;
    template<class Executor> const Executor* target() const noexcept;
};

// executor comparisons:

template <class... SupportableProperties>
bool operator==(const executor<SupportableProperties...>& a, const executor<SupportableProperties...>& b) noexcept;
template <class... SupportableProperties>
bool operator==(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template <class... SupportableProperties>
bool operator==(nullptr_t, const executor<SupportableProperties...>& e) noexcept;
template <class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& a, const executor<SupportableProperties...>& b) noexcept;
template <class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template <class... SupportableProperties>
bool operator!=(nullptr_t, const executor<SupportableProperties...>& e) noexcept;

// executor specialized algorithms:

template <class... SupportableProperties>
void swap(executor<SupportableProperties...>& a, executor<SupportableProperties...>& b) noexcept;

template <class Property, class... SupportableProperties>
executor prefer(const executor<SupportableProperties>& e, Property p);

```

The executor class satisfies the general requirements on executors.

[*Note*: To meet the `noexcept` requirements for executor copy constructors and move constructors, implementations may share a target between two or more executor objects. *--end note*]

The *target* is the executor object that is held by the wrapper.

1.5.2.1 executor constructors

```
executor() noexcept;
```

Postconditions: `!*this`.

```
executor(nullptr_t) noexcept;
```

Postconditions: `!*this`.

```
executor(const executor& e) noexcept;
```

Postconditions: `!*this` if `!e`; otherwise, `*this` targets `e.target()` or a copy of `e.target()`.

```
executor(executor&& e) noexcept;
```

Effects: If `!e`, `*this` has no target; otherwise, moves `e.target()` or move-constructs the target of `e` into the target of `*this`, leaving `e` in a valid state with an unspecified value.

```
template<class Executor> executor(Executor e);
```

Remarks: This function shall not participate in overload resolution unless: `* can_require_v<Executor, P, if P::is_requireable`, where `P` is each property in `SupportableProperties...` `* can_prefer_v<Executor, P, if P::is_preferable`, where `P` is each property in `SupportableProperties...` `* and can_query_v<Executor, P, where P is each property in SupportableProperties...`

Effects: `* *this` targets a copy of `e5` initialized with `std::move(e5)`, where: `* If CONTAINS_PROPERTY(execution::single_t, SupportableProperties), e1 is the result of execution::require(e, execution::single_t), otherwise e1 is e, * If CONTAINS_PROPERTY(execution::bulk_t, SupportableProperties), e2 is the result of execution::require(e, execution::bulk_t), otherwise e2 is e1 * If CONTAINS_PROPERTY(execution::oneway_t, SupportableProperties), e3 is the result of execution::require(e, execution::oneway_t), otherwise e3 is e2 * If CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties), e4 is the result of execution::require(e, execution::twoway_t), otherwise e4 is e3. * * If CONTAINS_PROPERTY(execution::then_t, SupportableProperties), e5 is the result of execution::require(e, execution::then_t), otherwise e5 is e4.`

```
template<class... OtherSupportableProperties> executor(executor<OtherSupportableProperties...> e);
```

Remarks: This function shall not participate in overload resolution unless: `* CONTAINS_PROPERTY(p, OtherSupportableProperties)`, where `p` is each property in `SupportableProperties...`

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

1.5.2.2 executor assignment

```
executor& operator=(const executor& e) noexcept;
```

Effects: `executor(e).swap(*this)`.

Returns: `*this`.

```
executor& operator=(executor&& e) noexcept;
```

Effects: Replaces the target of `*this` with the target of `e`, leaving `e` in a valid state with an unspecified value.

Returns: `*this`.

```
executor& operator=(nullptr_t) noexcept;
```

Effects: `executor(nullptr).swap(*this)`.

Returns: `*this`.

```
template<class Executor> executor& operator=(Executor e);
```

Requires: As for `template<class Executor> executor(Executor e)`.

Effects: `executor(std::move(e)).swap(*this)`.

Returns: `*this`.

1.5.2.3 executor destructor

```
~executor();
```

Effects: If `*this != nullptr`, releases shared ownership of, or destroys, the target of `*this`.

1.5.2.4 executor modifiers

```
void swap(executor& other) noexcept;
```

Effects: Interchanges the targets of `*this` and `other`.

1.5.2.5 executor operations

```
template <class Property>
executor require(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless: `CONTAINS_PROPERTY(Property, SupportableProperties)` && `Property::is_requirable`.

Returns: A polymorphic wrapper whose target is the result of `execution::require(e, p)`, where `e` is the target object of `*this`.

```
template <class Property>
executor query(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless: `CONTAINS_PROPERTY(Property, SupportableProperties)`.

Effects: Performs `execution::query(e, p)`, where `e` is the target object of `*this`.

Returns: `static_cast<Property::polymorphic_query_result_type>(e.query(p))`.

```
template<class Function>
void execute(Function&& f) const;
```

Remarks: This function shall not participate in overload resolution unless: `* CONTAINS_PROPERTY(execution::oneway_t, SupportableProperties)`, `*` and `CONTAINS_PROPERTY(execution::single_t, SupportableProperties)`.

Effects: Performs `e.execute(f2)`, where:

- `e` is the target object of `*this`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when called as `f2()`, performs `f1()`.

```
template<class Function>
std::experimental::future<result_of_t<decay_t<Function>()>>
twoway_execute(Function&& f) const
```

Remarks: This function shall not participate in overload resolution unless: `* CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties)`, `*` and `CONTAINS_PROPERTY(execution::single_t, SupportableProperties)`.

Effects: Performs `e.twoway_execute(f2)`, where:

- `e` is the target object of `*this`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when called as `f2()`, performs `f1()`.

Returns: A future, whose shared state is made ready when the future returned by `e.twoway_execute(f2)` is made ready, containing the result of `f1()` or any exception thrown by `f1()`. [Note: `e2.twoway_execute(f2)` may return any future type that satisfies the Future requirements, and not necessarily `std::experimental::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's `then()` member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

```
template<class Function, class SharedFactory>
void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Remarks: This function shall not participate in overload resolution unless: `* CONTAINS_PROPERTY(execution::oneway_t, SupportableProperties)`, `*` and `CONTAINS_PROPERTY(execution::bulk_t, SupportableProperties)`.

Effects: Performs `e.bulk_execute(f2, n, sf2)`, where:

- `e` is the target object of `*this`;

- sf1 is the result of DECAF_COPY(std::forward<SharedFactory>(sf));
- sf2 is a function object of unspecified type that, when called as sf2(), performs sf1();
- s1 is the result of sf1();
- s2 is the result of sf2();
- f1 is the result of DECAF_COPY(std::forward<Function>(f));
- f2 is a function object of unspecified type that, when called as f2(i, s2), performs f1(i, s1), where i is a value of type size_t.

```
template<class Function, class ResultFactory, class SharedFactory>
std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
    void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Remarks: This function shall not participate in overload resolution unless: * CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties), * and CONTAINS_PROPERTY(execution::bulk_t, SupportableProperties).

Effects: Performs e.bulk_twoway_execute(f2, n, rf2, sf2), where:

- e is the target object of *this;
- rf1 is the result of DECAF_COPY(std::forward<ResultFactory>(rf));
- rf2 is a function object of unspecified type that, when called as rf2(), performs rf1();
- sf1 is the result of DECAF_COPY(std::forward<SharedFactory>(rf));
- sf2 is a function object of unspecified type that, when called as sf2(), performs sf1();
- if decltype(rf1()) is non-void, r1 is the result of rf1();
- if decltype(rf2()) is non-void, r2 is the result of rf2();
- s1 is the result of sf1();
- s2 is the result of sf2();
- f1 is the result of DECAF_COPY(std::forward<Function>(f));
- if decltype(rf1()) is non-void and decltype(rf2()) is non-void, f2 is a function object of unspecified type that, when called as f2(i, r2, s2), performs f1(i, r1, s1), where i is a value of type size_t.
- if decltype(rf1()) is non-void and decltype(rf2()) is void, f2 is a function object of unspecified type that, when called as f2(i, s2), performs f1(i, r1, s1), where i is a value of type size_t.
- if decltype(rf1()) is void and decltype(rf2()) is non-void, f2 is a function object of unspecified type that, when called as f2(i, r2, s2), performs f1(i, s1), where i is a value of type size_t.
- if decltype(rf1()) is void and decltype(rf2()) is void, f2 is a function object of unspecified type that, when called as f2(i, s2), performs f1(i, s1), where i is a value of type size_t.

Returns: A future, whose shared state is made ready when the future returned by e.bulk_twoway_execute(f2, n, rf2, sf2) is made ready, containing the result in r1 (if decltype(rf1()) is non-void) or any exception thrown by an invocation f1. [Note: e.bulk_twoway_execute(f2) may return any future type that satisfies the Future requirements, and not necessarily std::experimental::future. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's then() member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

1.5.2.6 executor capacity

```
explicit operator bool() const noexcept;
```

Returns: true if *this has a target, otherwise false.

1.5.2.7 executor target access

```
const type_info& target_type() const noexcept;
```

Returns: If *this has a target of type T, typeid(T); otherwise, typeid(void).

```
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
```

Returns: If target_type() == typeid(Executor) a pointer to the stored executor target; otherwise a null pointer value.

1.5.2.8 executor comparisons

```
template<class... SupportableProperties>
bool operator==(const executor<SupportableProperties...> a, const executor<SupportableProperties...> b) noexcept;
```

Returns:

- true if !a and !b;
- true if a and b share a target;
- true if e and f are the same type and e == f, where e is the target of a and f is the target of b;
- otherwise false.

```
template<class... SupportableProperties>
bool operator==(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator==(nullptr_t, const executor<SupportableProperties...>& e) noexcept;
```

Returns: !e.

```
template<class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& a, const executor<SupportableProperties...>& b) noexcept;
```

Returns: !(a == b).

```
template<class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator!=(nullptr_t, const executor<SupportableProperties...>& e) noexcept;
```

Returns: (bool) e.

1.5.2.9 executor specialized algorithms

```
template<class... SupportableProperties>
void swap(executor<SupportableProperties...>& a, executor<SupportableProperties...>& b) noexcept;
```

Effects: a.swap(b).

```
template <class Property, class... SupportableProperties>
executor prefer(const executor<SupportableProperties...>& e, Property p);
```

Remarks: This function shall not participate in overload resolution unless: CONTAINS_PROPERTY(Property, SupportableProperties).

Returns: A polymorphic wrapper whose target is the result of execution::prefer(e, p), where e is the target object of *this.

1.5.3 Struct prefer_only

The prefer_only struct is a property adapter that disables the is_requirable value.

[Example:

Consider a generic function that performs some task immediately if it can, and otherwise asynchronously in the background.

```
template<class Executor>
void do_async_work(
    Executor ex,
    Callback callback)
{
    if (try_work() == done)
    {
        // Work completed immediately, invoke callback.
        execution::require(ex,
            execution::single,
            execution::oneway,
        ).execute(callback);
    }
    else
    {
        // Perform work in background. Track outstanding work.
        start_background_work(
            execution::prefer(ex,
                execution::outstanding_work),
            callback);
    }
}
```

This function can be used with an inline executor which is defined as follows:

```
struct inline_executor
{
    constexpr bool operator==(const inline_executor&) const noexcept
    {
```

```

        return true;
    }

constexpr bool operator!=(const inline_executor&) const noexcept
{
    return false;
}

template<class Function> void execute(Function f) const noexcept
{
    f();
}
};

```

as, in the case of an unsupported property, the `execution::prefer` call will fall back to an identity operation.

The polymorphic executor wrapper should be able to simply swap in, so that we could change `do_async_work` to the non-template function:

```

void do_async_work(
    executor<
        execution::single,
        execution::oneway,
        execution::outstanding_work> ex,
    std::function<void()> callback)
{
    if (try_work() == done)
    {
        // Work completed immediately, invoke callback.
        execution::require(ex,
            execution::single,
            execution::oneway,
        ).execute(callback);
    }
    else
    {
        // Perform work in background. Track outstanding work.
        start_background_work(
            execution::prefer(ex,
                execution::outstanding_work),
            callback);
    }
}

```

with no change in behavior or semantics.

However, if we simply specify `execution::outstanding_work` in the executor template parameter list, we will get a compile error. This is because the executor template doesn't know that `execution::outstanding_work` is intended for use with `prefer` only. At the point of construction from an `inline_executor` called `ex`, executor will try to instantiate implementation templates that perform the ill-formed `execution::require(ex, execution::outstanding_work)`.

The `prefer_only` adapter addresses this by turning off the `is_requirable` attribute for a specific property. It would be used in the above example as follows:

```

void do_async_work(
    executor<
        execution::single,
        execution::oneway,
        prefer_only<execution::outstanding_work>> ex,
    std::function<void()> callback)
{
    ...
}

```

-- end example]

```

template<class InnerProperty>
struct prefer_only
{
    InnerProperty property;

    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = InnerProperty::is_preferable;

    using polymorphic_query_result_type = see-below; // not always defined

    template<class Executor>

```

```

static constexpr auto static_query_v = see-below; // not always defined

constexpr prefer_only(const InnerProperty& p);

constexpr auto value() const
    noexcept(noexcept(std::declval<const InnerProperty>().value()))
    -> decltype(std::declval<const InnerProperty>().value());

template<class Executor>
friend auto prefer(Executor ex, const prefer_only& p)
    noexcept(noexcept(execution::prefer(std::move(ex), std::declval<const InnerProperty>()))
    -> decltype(execution::prefer(std::move(ex), std::declval<const InnerProperty>()));

template<class Executor>
friend constexpr auto query(const Executor& ex, const prefer_only& p)
    noexcept(noexcept(execution::query(ex, std::declval<const InnerProperty>())))
    -> decltype(execution::query(ex, std::declval<const InnerProperty>()));
};

```

If `InnerProperty::polymorphic_query_result_type` is valid and denotes a type, the template instantiation `prefer_only<InnerProperty>` defines a nested type `polymorphic_query_result_type` as a synonym for `InnerProperty::polymorphic_query_result_type`.

If `InnerProperty::static_query_v` is a variable template and `InnerProperty::static_query_v<E>` is well formed for some executor type `E`, the template instantiation `prefer_only<InnerProperty>` defines a nested variable template `static_query_v` as a synonym for `InnerProperty::static_query_v`.

```
constexpr prefer_only(const InnerProperty& p);
```

Effects: Initializes property with `p`.

```
constexpr auto value() const
    noexcept(noexcept(std::declval<const InnerProperty>().value()))
    -> decltype(std::declval<const InnerProperty>().value());
```

Returns: `property.value()`.

Remarks: Shall not participate in overload resolution unless the expression `property.value()` is well-formed.

```
template<class Executor>
friend auto prefer(Executor ex, const prefer_only& p)
    noexcept(noexcept(execution::prefer(std::move(ex), std::declval<const InnerProperty>())))
    -> decltype(execution::prefer(std::move(ex), std::declval<const InnerProperty>()));
```

Returns: `execution::prefer(std::move(ex), p.property)`.

Remarks: Shall not participate in overload resolution unless the expression `execution::prefer(std::move(ex), p.property)` is well-formed.

```
template<class Executor>
friend constexpr auto query(const Executor& ex, const prefer_only& p)
    noexcept(noexcept(execution::query(ex, std::declval<const InnerProperty>())))
    -> decltype(execution::query(ex, std::declval<const InnerProperty>()));
```

Returns: `execution::query(ex, p.property)`.

Remarks: Shall not participate in overload resolution unless the expression `execution::query(ex, p.property)` is well-formed.

1.6 Thread pools

Thread pools create execution agents which execute on threads without incurring the overhead of thread creation and destruction whenever such agents are needed.

1.6.1 Header `<thread_pool>` synopsis

```

namespace std {
namespace experimental {
inline namespace executors_v1 {

    class static_thread_pool;

} // inline namespace executors_v1
} // namespace experimental
} // namespace std

```

1.6.2 Class `static_thread_pool`

`static_thread_pool` is a statically-sized thread pool which may be explicitly grown via thread attachment. The `static_thread_pool` is expected to be created with the use case clearly in mind with the number of threads known by the creator. As a result, no default constructor is considered correct for arbitrary use cases and `static_thread_pool` does not support any form of automatic resizing.

`static_thread_pool` presents an effectively unbounded input queue and the execution functions of `static_thread_pool`'s associated executors do not block on this input queue.

[*Note:* Because `static_thread_pool` represents work as parallel execution agents, situations which require concurrent execution properties are not guaranteed correctness. --*end note.*]

```
class static_thread_pool
{
public:
    using executor_type = see-below;

    // construction/destruction
    explicit static_thread_pool(std::size_t num_threads);

    // nocopy
    static_thread_pool(const static_thread_pool&) = delete;
    static_thread_pool& operator=(const static_thread_pool&) = delete;

    // stop accepting incoming work and wait for work to drain
    ~static_thread_pool();

    // attach current thread to the thread pools list of worker threads
    void attach();

    // signal all work to complete
    void stop();

    // wait for all threads in the thread pool to complete
    void wait();

    // placeholder for a general approach to getting executors from
    // standard contexts.
    executor_type executor() noexcept;
};
```

For an object of type `static_thread_pool`, *outstanding work* is defined as the sum of:

- the number of existing executor objects associated with the `static_thread_pool` for which the `execution::outstanding_work` property is established;
- the number of function objects that have been added to the `static_thread_pool` via the `static_thread_pool` executor, but not yet executed; and
- the number of function objects that are currently being executed by the `static_thread_pool`.

The `static_thread_pool` member functions `executor`, `attach`, `wait`, and `stop`, and the associated executors' copy constructors and member functions, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

A `static_thread_pool`'s threads execute execution agents created via its associated executors with forward progress guarantee delegation. [*Note:* Forward progress is delegated to an execution agent for its lifetime. Because `static_thread_pool` guarantees only parallel forward progress to execution agents created via its executors, forward progress delegation does not apply to execution agents which have not yet started executing their first execution step. --*end note*]

1.6.2.1 Types

using executor_type = see-below;

An executor type conforming to the specification for `static_thread_pool` executor types described below.

1.6.2.2 Construction and destruction

```
static_thread_pool(std::size_t num_threads);
```

Effects: Constructs a `static_thread_pool` object with `num_threads` threads of execution, as if by creating objects of type `std::thread`.

```
~static_thread_pool();
```

Effects: Destroys an object of class `static_thread_pool`. Performs `stop()` followed by `wait()`.

1.6.2.3 Worker management

```
void attach();
```

Effects: Adds the calling thread to the pool such that this thread is used to execute submitted function objects. [*Note:* Threads created during thread pool construction, or previously attached to the pool, will continue to be used for function object execution. *--end note*] Blocks the calling thread until signalled to complete by `stop()` or `wait()`, and then blocks until all the threads created during `static_thread_pool` object construction have completed. (NAMING: a possible alternate name for this function is `join()`.)

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. The call to `stop()` returns without waiting for the threads to complete. Subsequent calls to `attach` complete immediately.

```
void wait();
```

Effects: If not already stopped, signals the threads in the pool to complete once the outstanding work is 0. Blocks the calling thread (C++Std [defns.block]) until all threads in the pool have completed, without executing submitted function objects in the calling thread. Subsequent calls to `attach` complete immediately.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `wait()` return.

1.6.2.4 Executor creation

```
executor_type executor() noexcept;
```

Returns: An executor that may be used to submit function objects to the thread pool. The returned executor has the following properties already established:

- `execution::oneway`
- `execution::twoway`
- `execution::then`
- `execution::single`
- `execution::bulk`
- `execution::possibly_blocking`
- `execution::not_continuation`
- `execution::not_outstanding_work`
- `execution::allocator`
- `execution::allocator(std::allocator<void>())`

1.6.3 static_thread_pool executor types

All executor types accessible through `static_thread_pool::executor()`, and subsequent calls to the member function require, conform to the following specification.

```
class C
{
public:
    // types:

    typedef std::size_t shape_type;
    typedef std::size_t index_type;

    // construct / copy / destroy:

    C(const C& other) noexcept;
    C(C&& other) noexcept;

    C& operator=(const C& other) noexcept;
    C& operator=(C&& other) noexcept;

    // executor operations:
```

```

see-below require(execution::never_blocking_t) const;
see-below require(execution::possibly_blocking_t) const;
see-below require(execution::always_blocking_t) const;
see-below require(execution::continuation_t) const;
see-below require(execution::not_continuation_t) const;
see-below require(execution::outstanding_work_t) const;
see-below require(execution::not_outstanding_work_t) const;
see-below require(const execution::allocator_t<void>& a) const;
template<class ProtoAllocator>
see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

static constexpr bool query(execution::bulk_parallel_execution_t) const;
static constexpr bool query(execution::thread_execution_mapping_t) const;
bool query(execution::never_blocking_t) const;
bool query(execution::possibly_blocking_t) const;
bool query(execution::always_blocking_t) const;
bool query(execution::continuation_t) const;
bool query(execution::not_continuation_t) const;
bool query(execution::outstanding_work_t) const;
bool query(execution::not_outstanding_work_t) const;
see-below query(execution::context_t) const noexcept;
see-below query(execution::allocator_t<void>) const noexcept;
template<class ProtoAllocator>
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;

bool running_in_this_thread() const noexcept;

template<class Function>
    void execute(Function&& f) const;

template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const

template<class Function, class Future>
    std::experimental::future<result_of_t<decay_t<Function>(decay_t<Future>)>>
        then_execute(Function&& f, Future&& pred) const;

template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
};

bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;

```

C is a type satisfying the OneWayExecutor, TwoWayExecutor, BulkOneWayExecutor, and BulkTwoWayExecutor requirements. Objects of type C are associated with a static_thread_pool.

1.6.3.1 Constructors

```
C(const C& other) noexcept;
```

Postconditions: *this == other.

```
C(C&& other) noexcept;
```

Postconditions: *this is equal to the prior value of other.

1.6.3.2 Assignment

```
C& operator=(const C& other) noexcept;
```

Postconditions: *this == other.

Returns: *this.

```
C& operator=(C&& other) noexcept;
```

Postconditions: *this is equal to the prior value of other.

Returns: *this.

1.6.3.3 Operations

```
see-below require(execution::never_blocking_t) const;
see-below require(execution::possibly_blocking_t) const;
see-below require(execution::always_blocking_t) const;
see-below require(execution::continuation_t) const;
see-below require(execution::not_continuation_t) const;
see-below require(execution::outstanding_work_t) const;
see-below require(execution::not_outstanding_work_t) const;
```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as **this*, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of **this*.

```
see-below require(const execution::allocator_t<void>& a) const;
```

Returns: `require(execution::allocator(x))`, where `x` is an implementation-defined default allocator.

```
template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;
```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as **this*, with the `execution::allocator_t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.alloc`. All other properties of the returned executor object are identical to those of **this*.

```
static constexpr bool query(execution::bulk_parallel_execution_t) const;
static constexpr bool query(execution::thread_execution_mapping_t) const;
```

Returns: `true`.

```
bool query(execution::never_blocking_t) const;
bool query(execution::possibly_blocking_t) const;
bool query(execution::always_blocking_t) const;
bool query(execution::continuation_t) const;
bool query(execution::not_continuation_t) const;
bool query(execution::outstanding_work_t) const;
bool query(execution::not_outstanding_work_t) const;
```

Returns: `true` if **this* has the property established such that it meets that properties requirements.

```
static_thread_pool& query(execution::context_t) const noexcept;
```

Returns: A reference to the associated `static_thread_pool` object.

```
see-below query(execution::allocator_t<void>) const noexcept;
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;
```

Returns: The allocator object associated with the executor, with type and value as either previously established by the `execution::allocator_t<ProtoAllocator>` property or the implementation defined default allocator established by the `execution::allocator_t<void>` property.

```
bool running_in_this_thread() const noexcept;
```

Returns: `true` if the current thread of execution is a thread that was created by or attached to the associated `static_thread_pool` object.

```
template<class Function>
    void execute(Function&& f) const;
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `OneWayExecutor` requirements and the properties established for **this*. If the submitted function `f` exits via an exception, the `static_thread_pool` calls `std::terminate()`.

```
template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `TwoWayExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `TwoWayExecutor` requirements.

```
template<class Function, class Future>
```

```
std::experimental::future<result_of_t<decay_t<Function>(decay_t<Future>)>>>
    then_execute(Function&& f, Future&& pred) const
```

Effects: Submits the function *f* for execution on the `static_thread_pool` according to the `ThenExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `ThenExecutor` requirements.

```
template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Effects: Submits the function *f* for bulk execution on the `static_thread_pool` according to the `BulkOneWayExecutor` requirements and the properties established for **this*. If the submitted function *f* exits via an exception, the `static_thread_pool` calls `std::terminate()`.

```
template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>>
        bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Effects: Submits the function *f* for bulk execution on the `static_thread_pool` according to the `BulkTwoWayExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `BulkTwoWayExecutor` requirements.

```
template<class Function, class Future, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>>
        bulk_then_execute(Function&& f, size_t n, Future&& pred, ResultFactory&& rf, SharedFactory&& sf) const;
```

Effects: Submits the function *f* for bulk execution on the `static_thread_pool` according to the `BulkThenExecutor` requirements and the properties established for **this*.

Returns: A future with behavior as specified by the `BulkThenExecutor` requirements.

1.6.3.4 Comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns: true if `&a.query(execution::context) == &b.query(execution::context)` and *a* and *b* have identical properties, otherwise false.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: `!(a == b)`.